

## model\_with\_word2Vec

December 2, 2024

```
[74]: # 1. Carregando as bibliotecas base
import numpy as np
import pandas as pd
import gensim

from sklearn import svm
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, recall_score, precision_score, f1_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
import xgboost as xgb

import seaborn as sns
from importlib import reload
import multiprocessing
import joblib

import scripts.preprocess as pp
import scripts.embeddings as eb
import scripts.convert as cv

reload(cv)
reload(pp)
reload(eb)
```

```
[74]: <module 'scripts.embeddings' from '/mnt/c/Users/luanm/projects/pece-monografia/src/models/scripts/embeddings.py'>
```

```
[75]: # 2. Carregar base de commits que será tratada

# Transformar blocos de código em commits
cv.convert_to_jsonl('./code_snippets', './inputs/commits.jsonl')

# Definir dataframe
df = pd.read_json('./inputs/commits.jsonl', lines=True)
df_labels = pd.read_csv('./inputs/labels.csv', sep=',')
```

```

labels = df_labels['Label']

print(f"Trecho de código com maior tamanho: {df['new_contents'].str.len().
      ↪min()}\n")
print(f"Trecho de código com maior tamanho: {df['new_contents'].str.len().
      ↪max()}\n")
print(f"Tamanho dos labels: {len(labels)}\n")

df.describe()

```

```

['GeneratedClass_1.cs', 'GeneratedClass_2.cs', 'GeneratedClass_3.cs',
'GeneratedClass_4.cs', 'GeneratedClass_5.cs', 'GeneratedClass_6.cs',
'GeneratedClass_7.cs', 'GeneratedClass_8.cs', 'GeneratedClass_9.cs',
'GeneratedClass_10.cs', 'GeneratedClass_11.cs', 'GeneratedClass_12.cs',
'GeneratedClass_13.cs', 'GeneratedClass_14.cs', 'GeneratedClass_15.cs',
'GeneratedClass_16.cs', 'GeneratedClass_17.cs', 'GeneratedClass_18.cs',
'GeneratedClass_19.cs', 'GeneratedClass_20.cs', 'GeneratedClass_21.cs',
'GeneratedClass_22.cs', 'GeneratedClass_23.cs', 'GeneratedClass_24.cs',
'GeneratedClass_25.cs', 'GeneratedClass_26.cs', 'GeneratedClass_27.cs',
'GeneratedClass_28.cs', 'GeneratedClass_29.cs', 'GeneratedClass_30.cs',
'GeneratedClass_31.cs', 'GeneratedClass_32.cs', 'GeneratedClass_33.cs',
'GeneratedClass_34.cs', 'GeneratedClass_35.cs', 'GeneratedClass_36.cs',
'GeneratedClass_37.cs', 'GeneratedClass_38.cs', 'GeneratedClass_39.cs',
'GeneratedClass_40.cs', 'GeneratedClass_41.cs', 'GeneratedClass_42.cs',
'GeneratedClass_43.cs', 'GeneratedClass_44.cs', 'GeneratedClass_45.cs',
'GeneratedClass_46.cs', 'GeneratedClass_47.cs', 'GeneratedClass_48.cs',
'GeneratedClass_49.cs', 'GeneratedClass_50.cs', 'GeneratedClass_51.cs',
'GeneratedClass_52.cs', 'GeneratedClass_53.cs', 'GeneratedClass_54.cs',
'GeneratedClass_55.cs', 'GeneratedClass_56.cs', 'GeneratedClass_57.cs',
'GeneratedClass_58.cs', 'GeneratedClass_59.cs', 'GeneratedClass_60.cs',
'GeneratedClass_61.cs', 'GeneratedClass_62.cs', 'GeneratedClass_63.cs',
'GeneratedClass_64.cs', 'GeneratedClass_65.cs', 'GeneratedClass_66.cs',
'GeneratedClass_67.cs', 'GeneratedClass_68.cs', 'GeneratedClass_69.cs',
'GeneratedClass_70.cs', 'GeneratedClass_71.cs', 'GeneratedClass_72.cs',
'GeneratedClass_73.cs', 'GeneratedClass_74.cs', 'GeneratedClass_75.cs',
'GeneratedClass_76.cs', 'GeneratedClass_77.cs', 'GeneratedClass_78.cs',
'GeneratedClass_79.cs', 'GeneratedClass_80.cs', 'GeneratedClass_81.cs',
'GeneratedClass_82.cs', 'GeneratedClass_83.cs', 'GeneratedClass_84.cs',
'GeneratedClass_85.cs', 'GeneratedClass_86.cs', 'GeneratedClass_87.cs',
'GeneratedClass_88.cs', 'GeneratedClass_89.cs', 'GeneratedClass_90.cs',
'GeneratedClass_91.cs', 'GeneratedClass_92.cs', 'GeneratedClass_93.cs',
'GeneratedClass_94.cs', 'GeneratedClass_95.cs', 'GeneratedClass_96.cs',
'GeneratedClass_97.cs', 'GeneratedClass_98.cs', 'GeneratedClass_99.cs',
'GeneratedClass_100.cs', 'GeneratedClass_101.cs', 'GeneratedClass_102.cs',
'GeneratedClass_103.cs', 'GeneratedClass_104.cs', 'GeneratedClass_105.cs',
'GeneratedClass_106.cs', 'GeneratedClass_107.cs', 'GeneratedClass_108.cs',

```

```
'GeneratedClass_109.cs', 'GeneratedClass_110.cs', 'GeneratedClass_111.cs',  
'GeneratedClass_112.cs', 'GeneratedClass_113.cs', 'GeneratedClass_114.cs',  
'GeneratedClass_115.cs']
```

Successfully converted 115 files to ./inputs/commits.jsonl

Trecho de código com maior tamanho: 0

Trecho de código com maior tamanho: 22251

Tamanho dos labels: 115

```
[75]:          new_contents  
count          115  
unique         108  
top  
freq           5
```

```
[76]: # 3. Preprocessar commits  
df['new_contents'] = df['new_contents'].apply(pp.clean_code)  
  
df.head()
```

```
[76]:          new_contents  
0  
1  <NEWLINE> using <LIBRARY>  <NEWLINE>  
2  
3          <ATTRIBUTE>  <NEWLINE>  
4    public  <VARIABLENAME>  <NEWLINE>
```

```
[77]: # 4. Escrever dataframe em csv para validação  
df.to_csv('./outputs/data_preprocessed.csv', sep=";", index=False)  
  
x_cleaned = df['new_contents']
```

```
[78]: # 5. Tokenização  
x_tokenized = [[c for c in s.split(" ") if c != ""] for s in x_cleaned]
```

```
[79]: # 5. Treinar gramática com Word2Vec  
  
# Número de cores da máquina  
cores = multiprocessing.cpu_count()  
  
print(f"Número de cores: {cores}")  
  
# Treinar o modelo  
model = gensim.models.Word2Vec(  
    vector_size=200,
```

```

sg=1,
workers=cores-3,
window=20,
seed=42,
alpha=0.01,
min_alpha=0.005)

model.build_vocab(x_tokenized, progress_per=10000)

model.train(x_tokenized, total_examples=model.corpus_count, epochs=20)

# Verificar similaridade
similarity_by_key_sample = model.wv.similar_by_key("public")
print(f"Exemplo de similaridade: + {similarity_by_key_sample}")

most_similar_sample = model.wv.similar_by_key("<METHOD_DEF:LENGHT_GTT:True>")
print(f"Exemplo de similaridade: + {most_similar_sample}")

# Número total de palavras do corpus
print(f"Número total de palavra do corpus: {model.corpus_total_words}")

```

Número de cores: 22

Exemplo de similaridade: + [('static', 0.9710027575492859), ('readonly', 0.965536892414093), ('private', 0.960656464099884), ('class', 0.9553068280220032), ('<CLASSNAME>', 0.9479354023933411), ('get;', 0.9467548727989197), ('protected', 0.9380561113357544), ('<METHOD\_DEF:LENGHT\_GTT:False>', 0.934917688369751), (':', 0.9297769069671631), ('set;', 0.9275515079498291)]

Exemplo de similaridade: + [('return', 0.9902473092079163), ('get', 0.9789010882377625), ('<CLASSMETHODINVOCATION>', 0.9636496901512146), ('false;', 0.9599233865737915), ('\$ ', 0.949687123298645), ('-', 0.9341035485267639), ('!=', 0.9328575134277344), ('true;', 0.9291867613792419), ('<OBJECTMETHODINVOCATION>', 0.9246307015419006), (' ');', 0.9141374826431274)]

Número total de palavra do corpus: 3618

[80]: *# 6. Criação dos vetores a partir do vocabulário*

```

sample_vec = eb.textToVector(model.wv, "<METHOD_DEF:LENGHT_GTT:True> ", 200, 20)

x_vecs = np.asarray([eb.textToVector(model.wv, " ".join(x), 200, 20) for x in
    ↪x_tokenized])

# Verificando a dimensão dos dados
print(x_vecs.shape)

# Reduzir dimensão para treinamento do modelo:
pca_model = PCA(n_components=35)
pca_model.fit(x_vecs)

```

```

cumsum = np.cumsum(pca_model.explained_variance_ratio_)
print("Taxa de variância: ", sum(pca_model.explained_variance_ratio_))
print(pca_model.explained_variance_ratio_)

d = np.argmax(cumsum >= 0.95) + 1
print(d)

x_vecs = pca_model.transform(x_vecs)
x_vecs.shape

```

(115, 4000)

Taxa de variância: 0.9518170618931796

```

[0.3334567  0.15500511 0.05836945 0.04482777 0.0403345  0.0314922
 0.02430851 0.0230475  0.02093823 0.01799807 0.01605612 0.01475143
 0.01409193 0.01248536 0.01209462 0.0115841  0.01092843 0.01036136
 0.00950771 0.00881527 0.00844655 0.00833689 0.00660039 0.0063686
 0.00587593 0.00566674 0.00535427 0.00499367 0.00489131 0.00476572
 0.00459373 0.00456862 0.00406002 0.00358711 0.00325315]

```

35

[80]: (115, 35)

```

[81]: vocabulary = list(model.wv.index_to_key)

model.save("./outputs/word2vec_model.model")

```

```

[82]: # 7. Treinar SVM com GridSearchCV
X_train, X_test, y_train, y_test = train_test_split(
    x_vecs,
    labels,
    test_size=0.3,
    random_state=44,
    stratify=labels)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

clf = svm.SVC()

param_grid = {
    'C': [0.1, 1, 10], # Regularização
    'kernel': ['linear', 'rbf'], # Função de kernel
    'gamma': ['scale', 'auto'], # Parâmetro do kernel
}

```

```

grid_search = GridSearchCV(estimator=clf, param_grid=param_grid, cv=2,
    ↳scoring='f1_macro')

# Treinando o modelo com Grid Search
grid_search.fit(X_train_scaled, y_train)

# Melhor combinação de hiperparâmetros encontrada
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test_scaled)

print(f"Melhores parâmetros: {grid_search.best_params_}")

print(classification_report(y_test, y_pred, zero_division=0))

```

```

Melhores parâmetros: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.81      | 0.96   | 0.88     | 26      |
| 1            | 0.00      | 0.00   | 0.00     | 4       |
| 2            | 1.00      | 0.50   | 0.67     | 2       |
| 3            | 0.67      | 0.67   | 0.67     | 3       |
| accuracy     |           |        | 0.80     | 35      |
| macro avg    | 0.62      | 0.53   | 0.55     | 35      |
| weighted avg | 0.71      | 0.80   | 0.75     | 35      |

```

[83]: # 8. Treinar RandomForest com GridSearchCV
X_train, X_test, y_train, y_test = train_test_split(x_vecs, labels, test_size=0.
    ↳3, random_state=44, stratify=labels)

# Instanciando o classificador Random Forest
rf = RandomForestClassifier(random_state=42)

# Definir os parâmetros para o GridSearch
param_grid = {
    'n_estimators': [10, 50, 100, 200],          # Número de árvores na floresta
    'max_depth': [None, 10, 20, 30],             # Profundidade máxima das árvores
    'min_samples_split': [2, 5, 10],             # Mínimo de amostras para
    ↳dividir um nó
    'min_samples_leaf': [1, 2, 4],               # Mínimo de amostras por folha
    'bootstrap': [True, False],                 # Usar amostragem bootstrap
}

# Configurar GridSearchCV
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid,

```

```

cv=2, scoring='f1_macro')

# Treinando com o GridSearch
grid_search.fit(X_train, y_train)

# Melhor combinação de hiperparâmetros encontrada
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

print(f"Melhores parâmetros: {grid_search.best_params_}")

print(classification_report(y_test, y_pred, zero_division=0))

```

Melhores parâmetros: {'bootstrap': False, 'max\_depth': None, 'min\_samples\_leaf': 1, 'min\_samples\_split': 5, 'n\_estimators': 10}

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.79      | 1.00   | 0.88     | 26      |
| 1            | 1.00      | 0.25   | 0.40     | 4       |
| 2            | 0.00      | 0.00   | 0.00     | 2       |
| 3            | 1.00      | 0.33   | 0.50     | 3       |
| accuracy     |           |        | 0.80     | 35      |
| macro avg    | 0.70      | 0.40   | 0.45     | 35      |
| weighted avg | 0.79      | 0.80   | 0.74     | 35      |

[84]: # 9. Treinar XGBoost com GridSearchCV

```

X_train, X_test, y_train, y_test = train_test_split(x_vecs, labels, test_size=0.
↳3, random_state=72, stratify=labels)

clf = xgb.XGBClassifier(eval_metric='mlogloss')

param_grid = {
    'n_estimators': [50, 100],          # Número de árvores
    'max_depth': [3, 9],                # Profundidade máxima das árvores
    'learning_rate': [0.01],            # Taxa de aprendizado
    'subsample': [0.8],                 # Subamostragem
    'colsample_bytree': [0.8],           # Colunas usadas em cada árvore
    'gamma': [0.1],                     # Regularização para reduzir
↳overfitting
}

# Configurar GridSearchCV
grid_search = GridSearchCV(
    estimator=clf, param_grid=param_grid,

```

```

        scoring='f1_macro', cv=2)

# Treinando com o GridSearch
grid_search.fit(X_train, y_train)

# Melhor combinação de hiperparâmetros encontrada
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

print(f"Melhores parâmetros: {grid_search.best_params_}")

print(classification_report(y_test, y_pred, zero_division=0))

```

Melhores parâmetros: {'colsample\_bytree': 0.8, 'gamma': 0.1, 'learning\_rate': 0.01, 'max\_depth': 3, 'n\_estimators': 50, 'subsample': 0.8}

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.81      | 1.00   | 0.90     | 26      |
| 1            | 1.00      | 0.25   | 0.40     | 4       |
| 2            | 1.00      | 0.50   | 0.67     | 2       |
| 3            | 1.00      | 0.33   | 0.50     | 3       |
| accuracy     |           |        | 0.83     | 35      |
| macro avg    | 0.95      | 0.52   | 0.62     | 35      |
| weighted avg | 0.86      | 0.83   | 0.79     | 35      |